Resolución — Taller SQL sobre HR 5 — Funciones de texto, número y fecha

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta, su resultado en HR y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR. Los resultados salen de ejecutarlas ahí.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — Código de empleado e iniciales

Nivel: Básico · **Tablas:** `employees`

Se pide. Para los empleados del departamento 90, generar un código con el formato `EMP-00100` (el `employee_id` con ceros a la izquierda hasta completar 5 cifras) y sus iniciales en mayúsculas.

Mostrar: código, nombre completo e iniciales, ordenados por identificador.

Idea. `LPAD(texto, largo, relleno)` completa por la izquierda hasta el largo indicado, y `SUBSTR(texto, 1, 1)` toma la primera letra.

Pasos.

1. `LPAD(e.employee_id, 5, '0')` convierte el número en texto y lo rellena con ceros hasta 5 caracteres.
2. `'EMP-' || ...` le pone el prefijo.
3. La inicial del nombre es `SUBSTR(e.first_name, 1, 1)` y la del apellido `SUBSTR(e.last_name, 1, 1)`; `UPPER` las pasa a mayúsculas.
4. Se filtra `department_id = 90` y se ordena por `employee_id`.

```
SELECT 'EMP-' || LPAD(e.employee_id, 5, '0') AS codigo,
       e.first_name || ' ' || e.last_name AS nombre,
       UPPER(SUBSTR(e.first_name, 1, 1) || SUBSTR(e.last_name, 1, 1))
         AS iniciales
FROM hr.employees e
WHERE e.department_id = 90
ORDER BY e.employee_id;
```

Resultado en HR. 3 filas y 3 columnas:

```
CODIGO     NOMBRE       INICIALES
---------  -----------  ---------
EMP-00100  Steven King  SK
EMP-00101  Neena Yang   NY
EMP-00102  Lex Garcia   LG
```

Ojo. `LPAD` recorta cuando el texto es más largo que el largo pedido: `LPAD('123456', 5, '0')` devuelve `12345`, no `123456`. Por eso conviene pedir un largo que sobre.

Ejercicio 2 — Código de área del teléfono

Nivel: Intermedio · **Tablas:** `employees`

Se pide. El teléfono de cada empleado tiene bloques separados por puntos, como `1.650.555.0137`. Contar cuántos empleados hay por código de área, que es el segundo bloque.

Mostrar: código de área y cantidad de empleados, de mayor a menor cantidad y, si empatan, por código.

Idea. `REGEXP_SUBSTR(texto, patrón, inicio, n)` devuelve la n-ésima coincidencia del patrón. El patrón `[^.]+` significa «una o más letras que no son un punto», es decir, un bloque entre puntos.

Pasos.

1. El patrón `[^.]+` encuentra cada bloque del teléfono.
2. `REGEXP_SUBSTR(e.phone_number, '[^.]+', 1, 2)` toma el segundo bloque.
3. Se agrupa por esa misma expresión y se cuenta con `COUNT(*)`.
4. Se ordena por cantidad y, en empate, por código.

```
SELECT REGEXP_SUBSTR(e.phone_number, '[^.]+', 1, 2) AS codigo_area,
       COUNT(*) AS empleados
FROM hr.employees e
GROUP BY REGEXP_SUBSTR(e.phone_number, '[^.]+', 1, 2)
ORDER BY empleados DESC, codigo_area;
```

Resultado en HR. 5 filas y 2 columnas:

```
CODIGO_AREA  EMPLEADOS
-----------  ---------
650                 45
1632                35
515                 21
590                  5
603                  1
```

Ojo. Los teléfonos del Reino Unido tienen un bloque menos (`44.1632.960005`), pero el segundo bloque sigue siendo el código de área (1632), y por eso los 35 empleados de allí quedan agrupados. Como el código es texto, el orden alfabético pondría el 1632 antes que el 515.

Variante. Sin expresiones regulares se puede hacer con `SUBSTR` e `INSTR`: se busca la posición del primer punto y la del segundo, y se corta lo que hay entre ellos.

```
SELECT SUBSTR(e.phone_number,
              INSTR(e.phone_number, '.', 1, 1) + 1,
              INSTR(e.phone_number, '.', 1, 2)
                - INSTR(e.phone_number, '.', 1, 1) - 1) AS codigo_area,
       COUNT(*) AS empleados
FROM hr.employees e
GROUP BY SUBSTR(e.phone_number,
                INSTR(e.phone_number, '.', 1, 1) + 1,
                INSTR(e.phone_number, '.', 1, 2)
                  - INSTR(e.phone_number, '.', 1, 1) - 1)
ORDER BY empleados DESC, codigo_area;
```

Ejercicio 3 — Salario por hora y funciones de redondeo

Nivel: Básico · **Tablas:** `employees`

Se pide. Para los empleados del departamento IT (`department_id` 60), calcular el valor de la hora suponiendo 160 horas al mes, y mostrarlo sin redondear, redondeado a 1 decimal, truncado a 1 decimal, hacia arriba y hacia abajo.

Mostrar: nombre, salario, valor por hora (`por_hora`), `redondeo`, `truncado`, `techo` y `piso`, de mayor a menor salario y, si empatan, por nombre.

Idea. `ROUND` redondea al valor más cercano, `TRUNC` corta sin redondear, `CEIL` sube al siguiente entero y `FLOOR` baja al anterior.

Pasos.

1. El valor por hora es `e.salary / 160`.
2. `ROUND(x, 1)` y `TRUNC(x, 1)` trabajan con un decimal.
3. `CEIL(x)` y `FLOOR(x)` devuelven enteros.
4. Se filtra el departamento 60 y se ordena por salario descendente y nombre.

```
SELECT e.first_name,
       e.salary,
       e.salary / 160 AS por_hora,
       ROUND(e.salary / 160, 1) AS redondeo,
       TRUNC(e.salary / 160, 1) AS truncado,
       CEIL(e.salary / 160) AS techo,
       FLOOR(e.salary / 160) AS piso
FROM hr.employees e
WHERE e.department_id = 60
ORDER BY e.salary DESC, e.first_name;
```

Resultado en HR. 5 filas y 7 columnas:

```
FIRST_NAME  SALARY  POR_HORA  REDONDEO  TRUNCADO  TECHO  PISO
----------  ------  --------  --------  --------  -----  ----
Alexander     9000     56.25      56.3      56.2     57    56
Bruce         6000      37.5      37.5      37.5     38    37
David         4800        30        30        30     30    30
Valli         4800        30        30        30     30    30
Diana         4200     26.25      26.3      26.2     27    26
```

Ojo. Con 56.25 la hora, `ROUND` a un decimal da 56.3 (el 5 final sube) y `TRUNC` da 56.2. Cuando el valor ya es entero, como los 30 de quienes ganan 4800, las cuatro funciones devuelven lo mismo.

Ejercicio 4 — Muestra para auditoría con MOD

Nivel: Básico · **Tablas:** `employees`

Se pide. Auditoría quiere revisar una de cada cinco personas. Seleccionar a los empleados cuyo `employee_id` sea múltiplo de 5.

Mostrar: identificador, nombre completo y salario, ordenados por identificador.

Idea. `MOD(a, b)` devuelve el resto de dividir a entre b. Un número es múltiplo de 5 cuando el resto es 0.

Pasos.

1. `MOD(e.employee_id, 5)` da el resto de dividir el identificador entre 5.
2. El filtro es `MOD(e.employee_id, 5) = 0`.
3. Se ordena por identificador.

```
SELECT e.employee_id,
       e.first_name || ' ' || e.last_name AS nombre,
       e.salary
FROM hr.employees e
WHERE MOD(e.employee_id, 5) = 0
ORDER BY e.employee_id;
```

Resultado en HR. 22 filas y 3 columnas; se muestran las primeras 6 filas:

```
EMPLOYEE_ID  NOMBRE          SALARY
-----------  --------------  ------
        100  Steven King      24000
        105  David Williams    4800
        110  John Chen         8200
        115  Alexander Khoo    3100
        120  Matthew Weiss     8000
        125  Julia Nayer       3200
```

Ojo. `MOD` sirve también para repartir filas en grupos rotativos: `MOD(employee_id, 3)` da 0, 1 o 2. Con negativos hay que cuidarse: en Oracle `MOD(-7, 3)` da -1 y no 2, porque el resultado toma el signo del dividendo.

Ejercicio 5 — Antigüedad al 31 de diciembre de 2018

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Calcular la antigüedad en años completos y en meses completos de cada empleado a la fecha de corte 31/12/2018, y mostrar a los diez más antiguos.

Mostrar: nombre completo, fecha de contratación (DD/MM/AAAA), años completos y meses completos, del más antiguo al más reciente y, si empatan, por apellido y nombre.

Idea. `MONTHS_BETWEEN(fecha_final, fecha_inicial)` devuelve los meses entre dos fechas, con decimales. Truncarlo da los meses completos, y dividirlo entre 12 antes de truncar da los años completos.

Pasos.

1. `MONTHS_BETWEEN(DATE '2018-12-31', e.hire_date)` mide los meses desde la contratación hasta el corte.
2. `TRUNC(... / 12)` da los años completos y `TRUNC(...)` los meses completos.
3. Se ordena por `hire_date` ascendente, que es antigüedad descendente, y por apellido y nombre.
4. `FETCH FIRST 10 ROWS ONLY` se queda con los diez primeros.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       TO_CHAR(e.hire_date, 'DD/MM/YYYY') AS contratacion,
       TRUNC(MONTHS_BETWEEN(DATE '2018-12-31', e.hire_date) / 12) AS anios,
       TRUNC(MONTHS_BETWEEN(DATE '2018-12-31', e.hire_date)) AS meses
FROM hr.employees e
ORDER BY e.hire_date, e.last_name, e.first_name
FETCH FIRST 10 ROWS ONLY;
```

Resultado en HR. 10 filas y 4 columnas; se muestran las primeras 6 filas:

```
NOMBRE           CONTRATACION  ANIOS  MESES
---------------  ------------  -----  -----
Lex Garcia       13/01/2011        7     95
Hermann Brown    07/06/2012        6     78
William Gietz    07/06/2012        6     78
Shelley Higgins  07/06/2012        6     78
Susan Jacobs     07/06/2012        6     78
Daniel Faviet    16/08/2012        6     76
```

Ojo. La fecha de corte se escribe en la consulta en lugar de usar SYSDATE: con SYSDATE el resultado cambiaría cada día y no podría compararse con el resultado esperado. Además, `MONTHS_BETWEEN` calcula la parte decimal como si todos los meses tuvieran 31 días, por eso se trunca.

Ejercicio 6 — Fechas escritas en español

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Escribir la fecha de contratación de los empleados del departamento 90 en español, con el formato «17 de junio de 2013», y el día de la semana en que ocurrió.

Mostrar: nombre completo, fecha escrita en español y día de la semana en minúsculas, ordenados por fecha de contratación.

Idea. `TO_CHAR` acepta un tercer argumento con el idioma: `'NLS_DATE_LANGUAGE=SPANISH'`. El texto fijo dentro del formato va entre comillas dobles.

Pasos.

1. El formato `'fmDD "de" month "de" YYYY'` arma «17 de junio de 2013»; `month` en minúsculas da el mes en minúsculas.
2. El modificador `fm` quita los ceros y los espacios de relleno.
3. `'fmday'` da el nombre del día de la semana.
4. El tercer argumento fija el idioma en español para las dos columnas.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       TO_CHAR(e.hire_date, 'fmDD "de" month "de" YYYY',
               'NLS_DATE_LANGUAGE=SPANISH') AS fecha,
       TO_CHAR(e.hire_date, 'fmday', 'NLS_DATE_LANGUAGE=SPANISH')
         AS dia_semana
FROM hr.employees e
WHERE e.department_id = 90
ORDER BY e.hire_date;
```

Resultado en HR. 3 filas y 3 columnas:

```
NOMBRE       FECHA                     DIA_SEMANA
-----------  ------------------------  ----------
Lex Garcia   13 de enero de 2011       jueves
Steven King  17 de junio de 2013       lunes
Neena Yang   21 de septiembre de 2015  lunes
```

Ojo. Sin el tercer argumento los nombres salen en el idioma de la sesión, que en esta base es el inglés (`June`, `Monday`). Y sin `fm`, el nombre del día se rellena con espacios hasta el ancho del día más largo: `lunes` ocuparía 9 caracteres.

Ejercicio 7 — Fin del período de prueba y primer lunes

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Los empleados contratados desde el 1 de enero de 2018 tienen un período de prueba de 3 meses. Mostrar la fecha en que terminó y la fecha de la inducción de cierre, que es el primer lunes posterior al fin de la prueba.

Mostrar: nombre completo, fecha de contratación, fin de la prueba y fecha de la inducción (DD/MM/AAAA), ordenados por fecha de contratación y, si empatan, por apellido y nombre.

Idea. `ADD_MONTHS(fecha, n)` suma meses calendario y `NEXT_DAY(fecha, 'día')` devuelve la primera fecha posterior que cae en ese día de la semana. `TO_DATE` convierte un texto en fecha con el formato indicado.

Pasos.

1. El filtro `e.hire_date >= TO_DATE('01/01/2018', 'DD/MM/YYYY')` deja a los contratados desde esa fecha.
2. `ADD_MONTHS(e.hire_date, 3)` es el fin de la prueba.
3. `NEXT_DAY(..., 'MONDAY')` da el primer lunes posterior a esa fecha.

4. Las tres fechas se muestran con TO_CHAR y se ordena por la fecha real.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       TO_CHAR(e.hire_date, 'DD/MM/YYYY') AS contratacion,
       TO_CHAR(ADD_MONTHS(e.hire_date, 3), 'DD/MM/YYYY') AS fin_prueba,
       TO_CHAR(NEXT_DAY(ADD_MONTHS(e.hire_date, 3), 'MONDAY'),
               'DD/MM/YYYY') AS induccion
FROM hr.employees e
WHERE e.hire_date >= TO_DATE('01/01/2018', 'DD/MM/YYYY')
ORDER BY e.hire_date, e.last_name, e.first_name;
```

Resultado en HR. 11 filas y 4 columnas; se muestran las primeras 6 filas:

```
NOMBRE            CONTRATACION  FIN_PRUEBA  INDUCCION
----------------  ------------  ----------  ----------
Charles Johnson   04/01/2018    04/04/2018  09/04/2018
Douglas Grant     13/01/2018    13/04/2018  16/04/2018
Mattea Marvins    24/01/2018    24/04/2018  30/04/2018
Eleni Zlotkey     29/01/2018    29/04/2018  30/04/2018
Girard Geoni      03/02/2018    03/05/2018  07/05/2018
Hazel Philtanker  06/02/2018    06/05/2018  07/05/2018
```

Ojo. NEXT_DAY devuelve siempre una fecha posterior: si la fecha ya es lunes, devuelve el lunes de la semana siguiente. El nombre del día tiene que estar en el idioma de la sesión, aquí `'MONDAY'`. Y ADD_MONTHS ajusta los fines de mes: `ADD_MONTHS(DATE '2018-01-31', 1)` es el 28 de febrero.

Ejercicio 8 — El primer mes proporcional

Nivel: Avanzado · **Tablas:** `employees`

Se pide. A quienes entraron en 2018 se les paga el primer mes de forma proporcional a los días trabajados, contando el día de ingreso. Calcular los días trabajados en el mes de ingreso y el pago proporcional del salario.

Mostrar: nombre completo, fecha de contratación (DD/MM/AAAA), días trabajados, días del mes y pago proporcional con dos decimales, ordenados por fecha de contratación y, si empatan, por apellido y nombre.

Idea. `LAST_DAY(fecha)` devuelve el último día del mes de esa fecha, y restar dos fechas da la diferencia en días.

Pasos.

1. Los días trabajados son `LAST_DAY(e.hire_date) - e.hire_date + 1`; el `+ 1` cuenta el día de ingreso.
2. Los días del mes son el día del último día: `EXTRACT(DAY FROM LAST_DAY(e.hire_date))`.
3. El pago proporcional es `salario * días trabajados / días del mes`, con `ROUND(..., 2)`.
4. El filtro es `EXTRACT(YEAR FROM e.hire_date) = 2018`.

```
SELECT e.first_name || ' ' || e.last_name AS nombre,
       TO_CHAR(e.hire_date, 'DD/MM/YYYY') AS contratacion,
       LAST_DAY(e.hire_date) - e.hire_date + 1 AS dias_trabajados,
       EXTRACT(DAY FROM LAST_DAY(e.hire_date)) AS dias_mes,
       ROUND(e.salary * (LAST_DAY(e.hire_date) - e.hire_date + 1)
             / EXTRACT(DAY FROM LAST_DAY(e.hire_date)), 2) AS pago_mes
FROM hr.employees e
WHERE EXTRACT(YEAR FROM e.hire_date) = 2018
ORDER BY e.hire_date, e.last_name, e.first_name;
```

Resultado en HR. 11 filas y 5 columnas; se muestran las primeras 6 filas:

```
NOMBRE           CONTRATACION  DIAS_TRABAJADOS  DIAS_MES  PAGO_MES
---------------  ------------  ---------------  --------  --------
Charles Johnson  04/01/2018                28        31      5600
Douglas Grant    13/01/2018                19        31   1593.55
Mattea Marvins   24/01/2018                 8        31   1858.06
Eleni Zlotkey    29/01/2018                 3        31   1016.13
Girard Geoni     03/02/2018                26        28      2600
Hazel Philtanker 06/02/2018                23        28   1807.14
```

Ojo. Restar dos fechas da días con decimales si alguna trae hora. Aquí todas las fechas de contratación vienen a medianoche, por eso la resta es un entero. Cuando hay hora, conviene aplicar TRUNC a la fecha antes de restar.

Ejercicio 9 — Contrataciones por día de la semana

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Contar cuántas personas fueron contratadas en cada día de la semana. Mostrar el día en español, abreviado.

Mostrar: día de la semana y cantidad de contratados, de mayor a menor cantidad y, si empatan, por día.

Idea. Se agrupa por la expresión que da el día de la semana. El idioma va como tercer argumento para que el resultado no dependa de la sesión.

Pasos.

1. `TO_CHAR(e.hire_date, 'DY', 'NLS_DATE_LANGUAGE=SPANISH')` da el día abreviado en español.
2. Se agrupa por esa misma expresión y se cuenta con `COUNT(*)`.
3. Se ordena por cantidad descendente y, en empate, por el día.

```
SELECT TO_CHAR(e.hire_date, 'DY', 'NLS_DATE_LANGUAGE=SPANISH') AS dia,
       COUNT(*) AS contratados
FROM hr.employees e
GROUP BY TO_CHAR(e.hire_date, 'DY', 'NLS_DATE_LANGUAGE=SPANISH')
ORDER BY contratados DESC, dia;
```

Resultado en HR. 7 filas y 2 columnas:

```
DIA  CONTRATADOS
---  -----------
JUE           25
VIE           18
MAR           16
LUN           14
MIÉ           13
SÁB           11
DOM           10
```

Ojo. No conviene agrupar por `TO_CHAR(fecha, 'D')`: ese número depende del territorio de la sesión. En esta base el domingo es 1 y el lunes 2. Y el orden alfabético de los días no es el de la semana: para ordenar de lunes a domingo habría que ordenar por un número calculado.

Ejercicio 10 — Etiqueta de comisión

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Construir una tabla con cada valor distinto de comisión que existe en la plantilla y su etiqueta: «Sin comisión» cuando es nulo, o el porcentaje como texto, por ejemplo «25 %».

Mostrar: valor de comisión (`commission_pct`) y etiqueta, ordenados por comisión con el nulo primero.

Idea. `NVL2(x, si_no_es_nulo, si_es_nulo)` elige entre dos resultados según x sea o no `NULL`. `DISTINCT` quita las filas repetidas.

Pasos.

1. `DISTINCT` deja una fila por cada combinación distinta de comisión y etiqueta.
2. Si hay comisión, la etiqueta es `TO_CHAR(e.commission_pct * 100) || ' %'`.
3. Si no la hay, `NVL2` devuelve el texto «Sin comisión».
4. `ORDER BY e.commission_pct NULLS FIRST` sube el nulo.

```
SELECT DISTINCT e.commission_pct,
       NVL2(e.commission_pct,
            TO_CHAR(e.commission_pct * 100) || ' %',
            'Sin comisión') AS etiqueta
FROM hr.employees e
ORDER BY e.commission_pct NULLS FIRST;
```

Resultado en HR. 8 filas y 2 columnas:

```
COMMISSION_PCT  ETIQUETA
--------------  ------------
        (null)  Sin comisión
           0.1  10 %
          0.15  15 %
           0.2  20 %
          0.25  25 %
           0.3  30 %
          0.35  35 %
           0.4  40 %
```

Ojo. No sirve `COALESCE(TO_CHAR(e.commission_pct * 100) || ' %', 'Sin comisión')`: en Oracle, concatenar `NULL` con un texto da el texto, así que el resultado es `' %'` y nunca `NULL`, y `COALESCE` no llega a «Sin comisión». Por eso se pregunta por la comisión misma, con `NVL2`.

Ejercicio 11 — Categoría del cargo con DECODE

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Clasificar a los empleados según el tipo de su cargo, que es lo que va después del guion bajo en `job_id`: PRES y VP son «Dirección»; MAN y MGR son «Gerencia»; REP es «Representantes»; cualquier otro tipo es «Operativo». Contar cuántos empleados hay en cada categoría.

Mostrar: categoría y cantidad de empleados, de mayor a menor cantidad.

Idea. `DECODE(expr, buscar1, resultado1, buscar2, resultado2, ..., por_defecto)` compara una expresión con una lista de valores. Es la versión de `CASE` propia de Oracle.

Pasos.

1. `SUBSTR(e.job_id, INSTR(e.job_id, '_') + 1)` toma lo que hay después del guion bajo.
2. `DECODE` traduce cada tipo a su categoría, y el último argumento cubre al resto.
3. Una vista en línea calcula la categoría de cada empleado.
4. La consulta externa agrupa por categoría y cuenta.

```
SELECT x.categoria,
       COUNT(*) AS empleados
FROM (SELECT DECODE(SUBSTR(e.job_id, INSTR(e.job_id, '_') + 1),
                    'PRES', 'Dirección',
                    'VP',   'Dirección',
                    'MAN',  'Gerencia',
                    'MGR',  'Gerencia',
                    'REP',  'Representantes',
                            'Operativo') AS categoria
      FROM hr.employees e) x
GROUP BY x.categoria
ORDER BY empleados DESC, x.categoria;
```

Resultado en HR. 4 filas y 2 columnas:

```
CATEGORIA       EMPLEADOS
--------------  ---------
Operativo              57
Representantes         33
Gerencia               14
Dirección               3
```

Ojo. DECODE solo compara por igualdad, y considera iguales dos NULL, a diferencia de =. Para rangos o desigualdades hace falta CASE.

Variante. La misma clasificación con CASE simple, que además es estándar y se entiende en otros motores:

```
SELECT x.categoria,
       COUNT(*) AS empleados
FROM (SELECT CASE SUBSTR(e.job_id, INSTR(e.job_id, '_') + 1)
               WHEN 'PRES' THEN 'Dirección'
               WHEN 'VP'   THEN 'Dirección'
               WHEN 'MAN'  THEN 'Gerencia'
               WHEN 'MGR'  THEN 'Gerencia'
               WHEN 'REP'  THEN 'Representantes'
               ELSE 'Operativo'
             END AS categoria
      FROM hr.employees e) x
GROUP BY x.categoria
ORDER BY empleados DESC, x.categoria;
```

Ejercicio 12 — Bono por antigüedad con tope

Nivel: Reto · **Tablas:** `employees`

Se pide. Todos los empleados reciben un bono anual de 1 % del salario por cada año completo de antigüedad al 31/12/2018, con un tope de 5 %. Mostrar a los diez con el bono más alto.

Mostrar: nombre completo, salario, años de antigüedad, porcentaje aplicado con el tope y bono con formato `$1,200.00`, de mayor a menor bono y, si empatan, por apellido.

Idea. `LEAST(a, b)` devuelve el menor de sus argumentos, y sirve para poner un tope (GREATEST es el contrario, para un mínimo). TO_CHAR con un modelo de formato convierte un número en texto con símbolo y separadores.

Pasos.

1. Una vista en línea calcula una sola vez los años completos, con MONTHS_BETWEEN y TRUNC.
2. `LEAST(x.anios, 5)` es el porcentaje: un punto por año, con tope de 5.
3. El bono es `salario * porcentaje / 100`, y TO_CHAR con el modelo `'FM$99,999.00'` lo escribe con símbolo, coma de miles y dos decimales.
4. Se ordena por el valor del bono y por apellido, y `FETCH FIRST 10 ROWS ONLY` deja los diez primeros.

```
SELECT x.nombre,
       x.salary,
       x.anios,
       LEAST(x.anios, 5) AS porcentaje,
       TO_CHAR(x.salary * LEAST(x.anios, 5) / 100, 'FM$99,999.00') AS bono
FROM (SELECT e.first_name || ' ' || e.last_name AS nombre,
             e.last_name,
             e.salary,
             TRUNC(MONTHS_BETWEEN(DATE '2018-12-31', e.hire_date) / 12)
               AS anios
      FROM hr.employees e) x
ORDER BY x.salary * LEAST(x.anios, 5) DESC, x.last_name
FETCH FIRST 10 ROWS ONLY;
```

Resultado en HR. 10 filas y 5 columnas; se muestran las primeras 6 filas:

```
NOMBRE           SALARY  ANIOS  PORCENTAJE  BONO
---------------  ------  -----  ----------  ---------
Steven King       24000      5           5  $1,200.00
Lex Garcia        17000      7           5  $850.00
Nancy Gruenberg   12008      6           5  $600.40
Shelley Higgins   12008      6           5  $600.40
John Singh        14000      4           4  $560.00
Den Li            11000      6           5  $550.00
```

Ojo. Si no se usara la vista en línea, el cálculo de los años se repetiría tres veces. Y ordenar por la columna bono sería un error: es texto y se ordenaría alfabéticamente, de modo que al ordenar de mayor a menor `$850.00` iría antes que `$1,200.00`; hay que ordenar por el número. Si el valor no cabe en los nueves del modelo, `TO_CHAR` devuelve una fila de #.